FULL-STACK DEVELOPMENT

Practical Learning Guide

From browser fundamentals to secure, collaborative applications

What this guide contains

Nine learning modules with explanations, practical examples, code snippets, realistic scenarios, common mistakes, knowledge checks, a 20-question practice assessment, answer explanations, and a quick revision sheet.

Assessment target: 70% to pass. Use the practice assessment to identify gaps, review the explanations, and retry until you can explain why each answer is correct.

Module 1 — How the Web Actually Works

From typing a URL to pixels on screen

Learning objectives

• Trace an HTTP request from browser to server and back.

• Explain DNS, HTTP, status codes, headers, and browser rendering.

• Diagnose common failures using the response and browser behaviour.

The request–response journey

When a person enters a URL, the browser parses it, checks whether it can reuse cached information, and resolves the domain through DNS if it needs an IP address. It then establishes a connection to the server—usually TCP plus TLS for HTTPS, or a QUIC-based connection for HTTP/3—sends an HTTP request, receives a response, and processes the returned resources. The browser parses HTML into a DOM tree, parses CSS into style information, calculates layout, paints pixels, and composites layers. JavaScript can modify the DOM and trigger more work.

An HTTP request

A request commonly contains a method, path, headers, and sometimes a body. GET usually retrieves a representation; POST submits data or asks the server to process something; PUT replaces a resource representation; PATCH partially updates it; DELETE requests removal. Headers carry metadata such as accepted formats, cookies, authorization, and content type. A request body is common for POST, PUT, and PATCH.

Status codes and headers

Status codes are grouped by their first digit: 1xx informational, 2xx success, 3xx redirection, 4xx client-side problem, and 5xx server-side failure. Common examples: 200 OK, 201 Created, 204 No Content, 301/302 redirect, 304 Not Modified, 400 Bad Request, 401 unauthenticated, 403 forbidden, 404 not found, 429 too many requests, and 500 Internal Server Error. Headers such as Content-Type, Cache-Control, Location, Authorization, and Set-Cookie describe the response or control client behaviour.

DNS and rendering

DNS maps a domain name such as example.com to an IP address, often through a resolver and cached records. DNS does not send the webpage itself. HTML defines document structure, CSS controls presentation, and JavaScript adds behaviour. The rendering engine turns document and style information into a visual result. Large images, blocking scripts, and expensive layout changes can delay the page becoming usable.

Worked scenario

A page shows “Not Found.” The browser reached a server and received an HTTP response, so DNS and connectivity may be working. A 404 suggests the requested path or resource was not found. A 500 instead points toward a server-side failure. If the page never connects and the browser reports a name-resolution error, investigate DNS before application routes.

Example / code to study

```
GET /products/42 HTTP/1.1
Host: shop.example
Accept: application/json
Authorization: Bearer <token>
```

Common mistakes to avoid

• Assuming DNS stores the page content; it resolves names to addresses.

• Treating every 4xx or 5xx status as the same error.

• Thinking the browser downloads only HTML; pages often request CSS, JavaScript, fonts, and images too.

Knowledge check

Q: Which layer translates a domain name to an IP address?
Answer: DNS.

Q: What does a 403 generally mean?
Answer: The server understood the request but refuses access.

Q: What does the rendering engine do?
Answer: It processes document and style information to display the page.

ROYSTIAN FOUNDATION

Module 2 — HTML & CSS Foundations

Semantic structure and layout that holds up on every screen

Learning objectives

• Choose meaningful semantic elements.

• Use Flexbox for one-dimensional layouts and Grid for two-dimensional layouts.

• Create responsive layouts with flexible sizing and media queries.

Semantic HTML

HTML describes meaning and structure. Use header for introductory content, nav for navigation, main for the page's primary content, section for a thematic group, article for a self-contained item, aside for related content, and footer for closing information. Use button for an action and anchor for navigation. Semantic elements improve accessibility, browser interpretation, maintainability, and keyboard/screen-reader navigation.

Flexbox

Flexbox lays items out along one main axis: row or column. The parent becomes a flex container with display:flex. Use justify-content to distribute items along the main axis, align-items across the cross axis, gap for spacing, flex-wrap when items may wrap, and flex:1 when items should share available space. Flexbox is excellent for navigation bars, button groups, and aligning content.

CSS Grid

Grid manages rows and columns together. Use display:grid and define columns with grid-template-columns. The repeat() and minmax() functions support adaptive tracks. For example, repeat(auto-fit, minmax(220px, 1fr)) creates as many cards as fit while keeping each card at least about 220 pixels wide.

Responsive design and media queries

Start with fluid layouts, relative units, max-width, flexible images, and wrapping. Add a media query when the content—not a particular device brand—needs a layout change. A breakpoint is not a magic standard; it is a point where the design stops working well. Test narrow, medium, and wide screens, keyboard navigation, zoom, and long text.

Accessibility and layout example

A card list should have meaningful headings, readable contrast, visible focus styles, and controls that work with a keyboard. Use alt text for informative images and empty alt text for purely decorative images. Avoid using layout-only divs where a semantic element expresses the purpose.

Example / code to study

```
<main class="cards">
  <article class="card">
    <h2>Learn HTML</h2>
    <p>Build a meaningful document structure.</p>
    <a href="/modules/html">Open module</a>
  </article>
</main>

.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
img { max-width: 100%; height: auto; }
```

Common mistakes to avoid

• Using div for every element, even when nav, main, button, or article is more accurate.

• Choosing Flexbox or Grid by habit rather than by the layout problem.

• Designing only for a desktop width and adding many brittle breakpoints afterward.

Knowledge check

Q: Which layout model is best suited to rows and columns together?
Answer: CSS Grid.

Q: Which element is intended for an action?
Answer: A button.

Q: What should guide a breakpoint choice?
Answer: The point at which the content or layout needs to change.

ROYSTIAN FOUNDATION

Module 3 — JavaScript Essentials

The language behaviours that cause many real bugs

Learning objectives

• Use scope and closures deliberately.

• Explain the event loop, promises, and async/await.

• Update arrays and objects immutably.

Scope and closures

Scope determines where a variable can be accessed. let and const are block-scoped; var is function-scoped. A closure occurs when a function retains access to variables from the lexical scope where it was created. Closures are useful for callbacks, private state, and event handlers, but can surprise developers when a loop variable is captured incorrectly.

The event loop

JavaScript normally runs one piece of synchronous code at a time on its main thread. Asynchronous APIs can complete later. Promise callbacks (microtasks) are generally processed before timer callbacks (tasks/macrotasks) after the current call stack empties. Therefore, synchronous logs appear first, then resolved promise callbacks, then a zero-delay timer in a common example.

Promises and async/await

A promise represents eventual completion or failure. It can be pending, fulfilled, or rejected. Use then/catch/finally or async/await with try/catch. Await pauses the surrounding async function—not the entire JavaScript thread. Always handle failures and show the user a meaningful error state.

Immutability

Immutability means creating a new array or object rather than mutating the existing one. It helps predictable UI updates and change detection. Spread syntax copies only one level, so nested structures need careful copying. Methods such as map, filter, and toSorted (where supported) can create new results; sort mutates the original array.

Worked example

If a list of tasks is shared with a UI, do not directly assign tasks[0].done = true and expect every state system to detect the change. Instead, create a new list with map and a new object for the changed task. For a fetch operation, represent loading, success, empty, and error states explicitly.

Example / code to study

```
const tasks = [
  { id: 1, title: "Read", done: false },
  { id: 2, title: "Practice", done: false }
];

const updated = tasks.map(task =>
  task.id === 1 ? { ...task, done: true } : task
);

async function loadProfile() {
  try {
    const response = await fetch("/api/profile");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return await response.json();
  } catch (error) {
    console.error("Profile load failed", error);
    throw error;
  }
```

```
}
```

Common mistakes to avoid

• Assuming await blocks all JavaScript execution.

• Forgetting to handle rejected promises.

• Mutating shared state and then wondering why the UI did not update.

• Assuming spread syntax deeply clones nested objects.

Knowledge check

Q: What does a closure retain?
Answer: Access to variables in its lexical scope.

Q: What is the purpose of try/catch around await?
Answer: To handle errors from the awaited operation.

Q: Does array.sort() mutate the array?
Answer: Yes, sort() mutates the original array.

Module 4 — Component-Based Frontends

Composing interfaces from reusable units

Learning objectives

• Split interfaces into focused components.

• Distinguish props from state.

• Place shared state at the nearest common parent and manage side effects safely.

Components and single responsibility

A component is a reusable unit that receives inputs and renders part of an interface. A component should have a clear purpose: a product card displays one product; a search box collects a query; a results list displays matches. Smaller focused components are easier to test and reuse, but splitting every line into a component can create needless complexity.

Props versus state

Props are inputs provided by a parent and should be treated as read-only by the child. State is data owned by a component that can change over time and trigger a re-render. Examples of props include a product title or a callback. Examples of state include the currently selected tab, form input, or whether a modal is open.

Lifting state up

When two sibling components need the same changing value, move that value to their nearest common parent and pass it down through props. For example, a search input and results list should use the same query state so that typing updates the displayed results.

Effects and side effects

A side effect interacts with something outside the render calculation, such as fetching data, subscribing to an event, or updating the document title. In React, effects should declare the reactive values they use and clean up subscriptions when necessary. Avoid using effects for values that can be calculated directly from existing props or state. Incorrect dependencies can cause stale data or repeated requests.

Practical design

For a course dashboard, separate CourseCard, CourseGrid, SearchBar, ProgressSummary, and DashboardPage. Keep the course list in the parent if search and filters affect several children. Keep a modal’s open/closed state local if no other component needs it.

Example / code to study

```
function CourseCard({ title, progress }) {
  return (
    <article className="course-card">
      <h2>{title}</h2>
      <progress value={progress} max="100" />
      <p>{progress}% complete</p>
    </article>
  );
}

// Parent owns shared state:
function CoursePage() {
  const [query, setQuery] = useState("");
  // Filter courses from query, then pass results to child components.
}
```

Common mistakes to avoid

• Changing props inside a child.

• Duplicating the same shared state in multiple components.

• Using an effect for a simple derived value, causing extra renders or loops.

Knowledge check

Q: Are props intended to be changed by the receiving child?
Answer: No; treat props as read-only inputs.

Q: Where should shared state live?
Answer: At the nearest common ancestor of components that need it.

Q: Name one reason an effect needs cleanup.
Answer: To remove event listeners or subscriptions and avoid leaks or duplicate handling.

ROYSTIAN FOUNDATION

Module 5 — APIs & Data Fetching

Talking to a server without breaking the interface

Learning objectives

• Design resource-oriented REST endpoints.

• Handle loading, empty, success, and error states.

• Understand JSON, idempotency, caching, and duplicate requests.

REST and resources

REST-style APIs expose resources through URLs and use HTTP methods to express operations. For example, GET /api/courses retrieves courses, GET /api/courses/12 retrieves one course, POST /api/courses creates one, PATCH /api/courses/12 updates selected fields, and DELETE /api/courses/12 requests deletion. Consistent resource names make APIs easier to understand.

Endpoints and contracts

An endpoint is the combination of a route and the supported method, together with the expected request and response shape. Document required fields, status codes, authentication, pagination, and error formats. Do not make the client guess whether a missing result is an empty array, null, or an error.

JSON

JSON represents objects, arrays, strings, numbers, booleans, and null. It does not directly represent functions, undefined, or dates as a special type; dates are typically serialized as strings. Parse a response with response.json() and validate assumptions before rendering.

Idempotency and caching

An operation is idempotent when repeating the same request has the same intended effect as performing it once. GET, PUT, and DELETE are defined as idempotent in HTTP semantics, though repeated DELETE requests can return different status codes. POST is not generally idempotent. For payments or order creation, use an idempotency key when the API supports it so a retry does not create duplicate charges or orders. Caching can reduce latency and server load, but data freshness and cache-control rules matter.

UI states and duplicate requests

A reliable interface distinguishes loading, success with data, success with no results, and failure. Disable or guard duplicate submissions where appropriate, cancel stale requests when a user changes search terms, and avoid triggering fetches repeatedly from render logic. Use pagination for large collections and cache responses when safe.

Example / code to study

```
// Example API contract
GET /api/courses
200 OK
Content-Type: application/json

[
  { "id": 12, "title": "Web Basics", "progress": 40 }
]

// Client should handle failure and empty data.
const response = await fetch("/api/courses");
if (!response.ok) throw new Error("Could not load courses");
const courses = await response.json();
```

Common mistakes to avoid

• Assuming fetch rejects automatically for HTTP 404 or 500; check response.ok.

• Using POST retries for payments without a safe retry strategy.

• Rendering an endless spinner when the response is an empty array.

• Returning inconsistent response shapes for similar endpoints.

Knowledge check

Q: Which method is commonly used to retrieve a resource?
Answer: GET.

Q: Does fetch automatically throw for a 404 response?
Answer: No; check response.ok or status.

Q: Why use an idempotency key for a payment request?
Answer: To help ensure retries do not create duplicate effects.

ROYSTIAN FOUNDATION

Module 6 — Server-Side Fundamentals

Routing, middleware, and where business logic belongs

Learning objectives

• Organise a server around routes and handlers.

• Use middleware for shared concerns.

• Validate incoming data and protect secrets with environment variables.

Routes and handlers

A route matches a method and path, such as POST /api/enrollments. A route handler validates the request, calls the appropriate business logic or service, and returns a response with a suitable status code. Keep HTTP concerns—headers, parsing, status codes—separate from business rules when the application grows.

Middleware

Middleware runs in the request/response pipeline before or after route handlers. Typical uses include request logging, authentication, rate limiting, CORS handling, and attaching a request ID. Middleware should be predictable and should not accidentally block valid requests or expose sensitive data in logs.

Input validation

Treat all client input as untrusted, even when a frontend form has validation. Validate required fields, data types, lengths, allowed values, and business constraints on the server. Reject invalid input with a clear 4xx response. Use parameterised database queries or a safe ORM to avoid SQL injection.

Environment variables and configuration

Environment variables provide runtime configuration such as database URLs, API keys, and environment mode. Keep secrets out of source control and browser bundles. A variable with a NEXT_PUBLIC_ prefix in a Next.js client build is exposed to the browser; it must not contain a secret. Rotate exposed credentials and use least-privilege permissions.

Request lifecycle example

A request enters logging middleware, passes authentication, is rate-limited, reaches a route handler, is validated, invokes business logic, reads or writes data, and returns a structured response. Errors should be handled centrally where possible, without returning stack traces or secrets to users.

Example / code to study

```
// Framework-neutral handler sketch
async function createEnrollment(req, res) {
  const { courseId, userId } = req.body ?? {};

  if (!Number.isInteger(courseId) || courseId <= 0 ||
      typeof userId !== "string" || userId.length === 0) {
    return res.status(400).json({ error: "Invalid input" });
  }

  // Check permissions and business rules before writing.
  const enrollment = await enrollmentService.create({ courseId, userId });
  return res.status(201).json(enrollment);
}
```

Common mistakes to avoid

• Trusting client-side validation as the only validation.

• Putting secrets in frontend JavaScript or a public environment variable.

• Returning internal exception details to the user.

• Mixing all business logic into a large route handler.

Knowledge check

Q: What is middleware useful for?
Answer: Shared pipeline behaviour such as authentication, logging, and rate limiting.

Q: Where should untrusted input be validated?
Answer: At the server boundary before business logic uses it.

Q: Should a private database password be bundled into browser code?
Answer: Never.

ROYSTIAN FOUNDATION

Module 7 — Databases & Data Modelling

Designing storage you will not regret in six months

Learning objectives

• Model entities and relationships in a relational schema.

• Compare relational and document databases.

• Explain normalisation, foreign keys, indexes, and transactions.

Entities and relationships

An entity is a thing the application stores, such as Student, Course, or Enrollment. A primary key uniquely identifies a row. A foreign key references a row in another table. A many-to-many relationship, such as students enrolling in many courses, is commonly represented by a junction table Enrollment(student_id, course_id, enrolled_at).

Normalisation

Normalisation reduces duplicated facts and update anomalies. If a course title is repeated in every enrollment row, changing the title requires updating many records. Store course details once in Course and refer to the course using a foreign key. Normalisation is a design tool, not a rule to split data endlessly; denormalisation may be chosen later for measured performance reasons.

Relational versus document storage

Relational databases are strong when relationships, constraints, transactions, and SQL queries are important. Document databases store nested documents and can fit flexible or aggregate-shaped data. Choose based on access patterns, consistency needs, relationship complexity, and operational skills—not on trend alone.

Indexes

An index is an auxiliary structure that can speed up filtering, sorting, and joins. For example, an index on enrollments.student_id can help retrieve a student's enrollments. Indexes use storage and make writes more expensive because the index must be maintained. Inspect query plans and index columns used frequently in filters or joins.

Transactions and ACID

A transaction groups changes into one logical unit. ACID means Atomicity (all or nothing), Consistency (rules remain satisfied), Isolation (concurrent operations are controlled), and Durability (committed data survives failures under the database's guarantees). A money transfer should debit one account and credit another in the same transaction.

Schema example

Students(student_id, name, email), Courses(course_id, title), and Enrollments(student_id, course_id, enrolled_at) avoid duplicating course names in each enrollment. A unique constraint can prevent duplicate enrollment pairs, while foreign keys prevent references to nonexistent students or courses.

Example / code to study

```
CREATE TABLE Enrollments (
  student_id INTEGER NOT NULL REFERENCES Students(student_id),
  course_id  INTEGER NOT NULL REFERENCES Courses(course_id),
  enrolled_at TIMESTAMP NOT NULL,
  PRIMARY KEY (student_id, course_id)
);

CREATE INDEX idx_enrollments_course
ON Enrollments(course_id);
```

Common mistakes to avoid

- Using an index on every column without considering write/storage cost.
- Storing comma-separated lists where relational links are needed.
- Assuming a transaction means every query is automatically fast.
- Forgetting constraints that prevent invalid relationships or duplicate records.

Knowledge check

Q: What enforces a link to an existing student row?
Answer: A foreign key.

Q: Why normalise course details?
Answer: To reduce duplication and update anomalies.

Q: What does atomicity mean?
Answer: A transaction's operations all succeed together or are rolled back.

Module 8 — Authentication & Security Basics

Keeping accounts and data safe by default

Learning objectives

• Explain sessions and token-based authentication.

• Store credentials safely.

• Recognise common web vulnerabilities and safer defaults.

Authentication versus authorisation

Authentication asks who the user is; authorisation asks what the user may do. A logged-in learner may view their own progress but must not be allowed to approve volunteers or edit another learner's account unless explicitly authorised. Enforce permissions on the server for every sensitive operation.

Password hashing

Do not store plaintext passwords or reversible encryption of passwords. Store a salted password hash produced by a password-hashing algorithm designed to resist guessing, such as Argon2id, bcrypt, or scrypt, using suitable parameters. A salt prevents identical passwords from producing identical stored hashes. Do not invent a hashing scheme or use a fast general-purpose hash such as plain SHA-256 for password storage.

Sessions and access tokens

A session usually stores state on the server and gives the browser a session identifier, often in a cookie. An access token is a credential sent with requests; it may be a signed token with expiry and claims. Tokens need secure storage and validation, short lifetimes where appropriate, and careful revocation/refresh design. Cookies used for sessions should generally use Secure, HttpOnly, and an appropriate SameSite setting.

Cross-site scripting (XSS)

XSS happens when untrusted data is interpreted as executable script in a victim's browser. Avoid inserting untrusted strings as raw HTML, use framework escaping, sanitise HTML when rich text is genuinely needed, and deploy a suitable Content Security Policy as an additional layer. Do not rely on CSP as the only defence.

Other practical defences

Use HTTPS, least privilege, server-side access checks, CSRF protection where cookie-based authentication requires it, rate limits on login, safe error messages, dependency updates, and secure password-reset flows. Never put secret keys in frontend code. Log security-relevant events without logging passwords, tokens, or sensitive personal data.

Scenario

A user changes /api/users/15 to /api/users/16 and sees another person's profile. This is an authorisation failure, often called an insecure direct object reference. The server must verify that the requester is allowed to access the requested record; hiding the link in the interface is not sufficient.

Example / code to study

```
// Security checklist for a protected API route
// 1. Validate the session/token.
// 2. Load the requested resource.
// 3. Check the user's permission for that resource.
// 4. Validate the submitted fields.
// 5. Perform the operation and audit it safely.
// Never trust a user ID supplied by the browser alone.
```

Common mistakes to avoid

• Confusing authentication with authorisation.

• Storing plaintext passwords or using fast hashes for password storage.

• Assuming a hidden button is a security control.

• Rendering user-provided HTML without sanitisation.

Knowledge check

Q: What is the difference between authentication and authorisation?
Answer: Authentication identifies the user; authorisation checks permissions.

Q: Which cookie flag helps prevent JavaScript from reading a session cookie?
Answer: HttpOnly.

Q: How should passwords be stored?
Answer: As salted hashes from a password-hashing algorithm such as Argon2id, bcrypt, or scrypt.

Module 9 — Version Control & Collaboration

Working in a team without losing work

Learning objectives

• Use branches, commits, and pull requests effectively.

• Resolve merge conflicts methodically.

• Write useful commit messages and collaborate safely.

Git fundamentals

A repository tracks project history. A commit records a snapshot of staged changes and metadata. A branch is a movable name pointing to a line of development. A remote such as origin refers to a hosted repository. Git is distributed: each clone contains local history, and pushing shares commits with a remote.

A safe feature workflow

Start by updating the base branch, create a focused feature branch, make small changes, run tests, commit with an intent-focused message, push the branch, and open a pull request. Reviewers can discuss design and catch defects before the changes enter the main branch.

Commits and messages

A useful commit message describes intent, for example, 'Validate course enrollment input' rather than 'changes'. Small commits are easier to review and revert. Do not commit secrets, generated build output unless required, or unrelated formatting changes mixed with functional work.

Merge conflicts

A conflict occurs when Git cannot automatically combine overlapping edits. Open the conflicted file, inspect the base and both changes, choose or combine the correct behaviour, remove conflict markers, run tests, stage the resolved file, and complete the merge or rebase. Do not blindly choose 'ours' or 'theirs' without understanding the code.

Useful commands

git status shows the working tree; git diff shows unstaged changes; git add stages changes; git commit records them; git switch -c creates and switches to a branch; git pull fetches and integrates remote updates; git push publishes commits. Before a risky operation, inspect status and ensure uncommitted work is safe.

Team example

Two developers edit the same validation function. A merge conflict is not itself a failure; it is a signal that the combined intent needs human review. Discuss the expected behaviour, reconcile the code, add a regression test, and explain the resolution in the pull request.

Example / code to study

```
git switch main
git pull
git switch -c feature/course-search
# edit files and run tests
git status
git diff
git add src/
git commit -m "Add course search validation"
git push -u origin feature/course-search
```

Common mistakes to avoid

• Working directly on the main branch for every change.

• Committing credentials or API keys.

• Resolving a conflict by deleting markers without checking the logic.

• Opening a pull request without explaining what changed or how it was tested.

Knowledge check

Q: What is the purpose of a branch?
Answer: To isolate a line of development from other work.

Q: What is a pull request used for?
Answer: Reviewing and discussing proposed changes before integration.

Q: What should happen after resolving a conflict?
Answer: Review the combined logic, run tests, stage the result, and complete the merge/rebase.

ROYSTIAN FOUNDATION

Practice Assessment — 20 Questions

Choose the single best answer for each question. Suggested pass mark: 14/20 (70%). Try the questions before reading the explanations.

1. Which system resolves a domain name to an IP address?

A. CSS

B. DNS

C. DOM

D. JSON

2. A response status of 404 usually means:

A. The resource was not found

B. Authentication succeeded

C. The server is always offline

D. The request was redirected

3. Which HTML element is best for a user action such as submitting a form?

A. span

B. button

C. div

D. aside

4. Which CSS tool is designed for two-dimensional rows and columns?

A. Grid

B. Float only

C. Inline style

D. z-index

5. What is a closure?

A. A CSS breakpoint

B. A retained lexical environment accessible to a function

C. A database index

D. An HTTP redirect

6. What does await do inside an async function?

A. Stops all JavaScript globally

B. Waits for a promise result within that function

C. Converts every error into success

D. Makes an operation synchronous on the server

7. Which operation creates a new array of transformed elements?

A. map()

B. push()

C. sort()

D. reverse()

8. Props in a component are best treated as:

A. Read-only inputs

B. Global database records

C. Browser cookies

D. Always mutable

9. Two sibling components need the same changing search query. Where should it usually live?

A. In duplicated local state in each sibling

B. In the nearest common parent

C. In CSS

D. In an image file

10. Why check response.ok after fetch?

A. fetch rejects automatically for all 4xx/5xx responses

B. HTTP errors such as 404 usually still resolve to a Response

C. It converts JSON to HTML

D. It enables DNS

11. Which method is normally used to retrieve a resource?

A. GET

B. POST

C. CONNECT only

D. PATCH only

12. What is an idempotency key useful for in payment APIs?

A. Improving CSS

B. Preventing duplicate effects on safe retries

C. Replacing authentication

D. Encrypting the database

13. What should happen before server business logic processes client input?

A. Trust it because the browser validated it

B. Validate it on the server

C. Put it in a public variable

D. Skip type checks

14. Which value should not be exposed in frontend JavaScript?

A. Public page title

B. Private database password

C. Public image URL

D. Button label

15. What does a foreign key do?

A. Styles a table

B. References a row in another table and helps enforce relationships

C. Encrypts passwords

D. Speeds every query automatically

16. What is a common trade-off of indexes?

A. They always slow reads

B. They use storage and can add write overhead

C. They remove the need for a schema

D. They guarantee all queries are fast

17. Authentication asks:

A. What is this user's permission?

B. Who is this user?

C. How is CSS laid out?

D. Which branch is current?

18. Which is a safer password-storage approach?

A. Plain text

B. Base64 encoding

C. Salted Argon2id/bcrypt/scrypt hash

D. Reversible encryption with a hard-coded key

19. What is a pull request primarily for?

A. Deleting remote history automatically

B. Reviewing proposed changes before merging

C. Resolving DNS

D. Running HTML rendering

20. After resolving a Git merge conflict, what should you do?

A. Assume the code is correct

B. Review the result and run relevant tests

C. Delete the repository

D. Ignore conflict markers

Practice Assessment — Answer Key & Explanations

1. B — DNS resolves names to network addresses; it does not contain the page itself.

2. A — 404 is a client-error status indicating that the requested resource was not found.

3. B — Buttons represent actions and support keyboard interaction.

4. A — Grid manages both rows and columns.

5. B — Closures preserve access to variables in the scope where a function was created.

6. B — Await suspends the async function until the promise settles; other work can continue.

7. A — map returns a new array; push, sort, and reverse mutate their array.

8. A — A child should not mutate its props.

9. B — Lifting state to the nearest common parent keeps both siblings synchronised.

10. B — Network failures reject fetch, but HTTP error statuses generally require an explicit check.

11. A — GET requests a representation of a resource.

12. B — A supported idempotency key lets the server recognise retries of the same logical operation.

13. B — Client-side validation improves UX but cannot be trusted as a security boundary.

14. B — Anything shipped to the browser can be inspected by users.

15. B — Foreign keys help preserve referential integrity.

16. B — Indexes can speed up selected queries but require maintenance and storage.

17. B — Authentication establishes identity; authorisation determines allowed actions.

18. C — Use a password-hashing algorithm designed to resist guessing.

19. B — Pull requests provide a review and discussion point for proposed changes.

20. B — Conflict resolution needs semantic review and testing, not just removal of markers.

Quick Revision Sheet

Web: DNS resolves names; HTTP carries requests and responses; status codes communicate outcomes; the browser parses HTML/CSS and renders pixels.

HTML/CSS: Use semantic elements; Flexbox for one axis; Grid for rows and columns; use fluid sizing and content-driven breakpoints.

JavaScript: Closures retain lexical access; promise callbacks run asynchronously; use async/await with error handling; avoid unintended mutation.

Components: Components should have focused responsibilities; props are read-only inputs; state is changing owned data; lift shared state to the nearest common parent.

APIs: Model REST endpoints around resources; check response.ok; support loading, empty, success, and error states; use safe retry/idempotency strategies.

Server: Handlers process routes; middleware handles shared concerns; validate input at the server boundary; keep secrets out of client bundles.

Database: Normalise to reduce duplication; use primary/foreign keys; index measured query patterns; transactions protect multi-step changes.

Security: Authentication is identity; authorisation is permission; hash passwords with a suitable password-hashing algorithm; prevent XSS; use HTTPS and least privilege.

Git: Branches isolate work; commits record intent; pull requests enable review; conflicts require careful reconciliation and tests.

Final preparation tip

For each module, explain the concept aloud, predict what the example code does, identify one failure mode, and describe how you would test it. Understanding why an option is wrong is more useful than memorising a definition.

Module 10 — Deployment & Production Hygiene

Shipping, then keeping it healthy.

Learning objectives

• Describe build, preview and production environments.

• Explain continuous deployment and safe rollback.

• Apply basic performance budgets and diagnose regressions.

• Use logs, metrics, traces and error tracking to understand production behaviour.

1. Build, preview and production environments

A **build step** transforms source files into deployable assets. Depending on the stack, it may compile TypeScript, bundle JavaScript and CSS, optimise images, run static checks, and produce output ready to serve. A successful build proves that the project can be packaged; it does not prove that every user flow works correctly.

Development is where developers make changes and get fast feedback. A **preview/staging environment** lets a team test a release candidate before real users depend on it. **Production** is the live environment. Keep configuration separate across environments, and never expose production credentials in preview builds.

2. Build checks and release quality

A reliable release pipeline combines a reproducible build with linting, type checks, automated tests, dependency/security checks, and smoke tests. A smoke test verifies a few critical flows after deployment—for example, that the homepage loads, login works, and a key API responds. Test the behaviours users depend on, not just whether the server starts.

3. Continuous deployment

Continuous deployment (CD) automatically releases a change to production after it passes the agreed checks. A common pipeline is: pull request → install locked dependencies → lint/type-check → test → build → deploy preview → smoke test → merge/approval → production deploy → monitor. Continuous delivery keeps changes ready to release; continuous deployment automatically releases them when the pipeline succeeds.

Use small, reversible changes. Protect production branches, limit who can deploy, keep secrets in the platform's secret manager, and make deployment status visible to the team.

4. Performance budgets

A performance budget sets an explicit limit for things such as JavaScript bundle size, page weight, image size, load time or interaction latency. Budgets help teams notice regressions early. Optimise large images, compress assets, lazy-load content that is not immediately needed, remove unused code, cache static assets appropriately, and avoid unnecessary network requests.

Measure performance instead of guessing. A single lab score is useful, but real-user measurements can reveal slow devices, poor connections and routes that synthetic tests miss. Choose thresholds that match the product and user needs.

5. Rollback and safe releases

Rollback means returning to a previous known-good release when a new deployment causes unacceptable problems. Keep prior release artifacts available and know who can initiate a rollback. After rolling back, verify that error rates and user flows recover.

Database changes need special care: reverting application code does not automatically reverse a destructive database migration. Prefer backward-compatible, staged changes—for example, add a new column first, deploy code that can work with both versions, migrate data, and remove the old column only in a later release.

6. Observability: logs, metrics and traces

Observability helps you infer what a running system is doing from its outputs. The three common signals are:

• **Logs:** individual events and errors, ideally with timestamps, severity, request IDs and useful context.

• **Metrics:** numerical measurements over time, such as request volume, error rate, latency, CPU and memory use.

• **Traces:** the path of one request across services, helping identify which step is slow or failing.

These signals complement each other. A metric can show that latency increased; logs may show an exception; a trace can reveal which service or database call caused the delay. Do not log passwords, access tokens, private keys or unnecessary personal data.

7. Error tracking and incident response

Error tracking captures exceptions and useful context such as stack traces, release versions and affected routes. Group repeated errors, prioritise by user impact, and alert on meaningful symptoms such as a sharp increase in 5xx responses or latency. Alerts should be actionable rather than noisy.

A practical incident sequence is: **detect** → **assess impact** → **mitigate or roll back** → **verify recovery** → **investigate root cause** → **add prevention**. After an incident, document what happened and add a test or monitoring rule where it can prevent or detect recurrence.

Worked scenario

A new release goes live and checkout starts returning HTTP 500 errors. Check the deployment timeline and error-rate metric, inspect logs for the failing route, and follow traces to find the failing dependency. If the release is the likely cause and customer impact is high, roll back if safe. Verify that checkout and error rates recover, then reproduce the problem in a safe environment and add a regression test.

Example deployment pipeline

1. Pull request opened and reviewed.
2. Install locked dependencies.
3. Run lint, type checks, tests and dependency checks.
4. Build optimised assets.
5. Deploy a preview and run smoke tests.
6. Merge approved changes and deploy production.
7. Check health metrics, logs, traces and error tracking.
8. Roll back if a serious regression occurs; investigate afterward.

Common mistakes

• Assuming a successful build guarantees correct behaviour.

• Deploying without tests, smoke checks or a rollback plan.

• Using logs alone while ignoring metrics and traces.

• Logging credentials or personal information.

• Reverting code without considering database migration compatibility.

• Setting performance targets without measuring actual user experience.

Knowledge check

Q: What is a preview environment for?
Answer: Testing a release candidate before it reaches production.

Q: What is rollback?
Answer: Returning to a previous known-good release.

Q: Name the three common observability signals.
Answer: Logs, metrics and traces.

Q: Why set a performance budget?
Answer: To make performance limits explicit and catch regressions early.

Q: Production errors rise sharply after deployment. What should you do?
Answer: Assess impact, inspect telemetry, roll back if appropriate, verify recovery, and then investigate the root cause.

Module 10 quick revision

Build packages and optimises the application. **Preview** validates a release candidate. **Production** serves real users. **Continuous deployment** releases changes automatically after checks. **Performance budgets** define acceptable limits. **Rollback** restores a known-good version. **Observability** combines logs, metrics and traces to help explain system behaviour.